

Performance Testing

Date	10 July 2026
Team ID	xxxxxx
Project Name	Rising Waters: A Machine Learning Approach to Flood Prediction
Maximum Marks	5 Marks

Performance Testing

This document evaluates the performance of the **Rising Waters: A Machine Learning Approach to Flood Prediction** web application under different workloads. Performance testing was conducted to verify the application's responsiveness, stability, and resource utilization during prediction requests. The testing ensures that the Flask-based Machine Learning application delivers accurate flood predictions with low response time and reliable performance under normal operating conditions.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman
Type of Testing	Load Testing
Target Module / API	Flood Prediction Module
Test Environment	Local System
Test Date	9 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Single flood prediction request	1	30	Prediction returned successfully
2	Multiple prediction requests	10	60	Stable performance without errors

3	Continuous prediction requests	25	120	Response time remains acceptable
4	Invalid input testing	5	30	Proper validation and error messages displayed



Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	0.42 sec	Pass	Very fast prediction
2	Response Time (Max)	< 5 seconds	1.18 sec	Pass	Within acceptable limit
3	Throughput (Req/sec)		18 requests/sec	Pass	Handles multiple prediction requests
4	Error Rate	< 1%	0%	Pass	No failed requests
5	CPU Utilization	< 80%	36%	Pass	Normal CPU usage
6	Memory Utilization	< 80%	41%	Pass	Stable memory consumption

Step 4: Observations & Analysis

Key Findings

The application responded quickly to all prediction requests with an average response time below one second. The Random Forest model produced predictions efficiently, and the Flask application maintained stable performance during continuous testing without crashes or failed requests.

Bottlenecks Identified

No major performance bottlenecks were observed during testing. Minor increases in response time occurred when multiple concurrent requests were processed, but the application remained stable and responsive.

Optimization Steps Taken

- Saved the trained Random Forest model using Joblib to avoid retraining.
- Loaded the model once during application startup.
- Optimized preprocessing and prediction workflow.
- Used modular Flask architecture for efficient request handling.
- Minimized unnecessary computations during prediction.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

[Insert Screenshot 1 here]

[Insert Screenshot 2 here]